

Mobile Application Development

Data Persistence with DataStore in Jetpack Compose



Store and Access Data Using DataStore

Why DataStore over SharedPreferences?

DataStore is the recommended modern alternative to SharedPreferences.

Addresses issues of SharedPreferences:

- Lack of type safety
- Synchronous APIs (can cause UI lag)
- No built-in support for asynchronous programming or flows

Preferences DataStore vs. Proto DataStore

Preferences DataStore 	Proto DataStore 
Stores primitive key-value pairs	Stores typed objects defined by a schema
Simpler setup	More structured and type-safe
Best for simple data	Best for structured, complex data
Easier migration from SharedPreferences	Ideal for large, structured data

Benefits of Using DataStore

- **Type Safety:** Strongly typed keys and data.
- **Asynchronous by default:** Uses Kotlin Coroutines and Flow.
- **Lifecycle-aware:** Prevents memory leaks.
- **Reactive UI updates:** Seamlessly integrates with Compose.

Setting up Preferences DataStore

- Add dependency:

```
implementation "androidx.datastore:datastore-preferences:1.2.1"
```

- Initialize DataStore in Android:

```
val Context.dataStore by preferencesDataStore("settings")
```

Creating and Using Keys

Define keys clearly and strongly typed:

```
val USERNAME_KEY = stringPreferencesKey("username")  
val AGE_KEY = intPreferencesKey("age")
```

Reading and Writing Data

Write Data:

```
context.dataStore.edit { settings ->
    settings[USERNAME_KEY] = "JohnDoe"
    settings[AGE_KEY] = 30
}
```

Read Data (Flow):

```
val usernameFlow: Flow<String> = context.dataStore.data.map { preferences ->
    preferences[USERNAME_KEY] ?: "Guest"
}
```


Observing Data Changes in Compose

Compose directly observes `DataStore` with `collectAsState()`:

```
val username by context.dataStore.data
    .map { prefs -> prefs[USERNAME_KEY] ?: "Guest" }
    .collectAsState(initial = "Loading...")
```

```
Text("Welcome, $username!")
```

- UI automatically updates when data changes.

Handling Default Values

- Always provide default values to avoid nulls and simplify UI logic.

```
val ageFlow = context.dataStore.data.map { prefs -> prefs[AGE_KEY] ?: 0 }
```

Integration with ViewModel

- ViewModel manages all data access, UI remains clean.

Example ViewModel:

```
class UserViewModel(private val context: Context) : ViewModel() {  
    val username = context.dataStore.data  
        .map { prefs -> prefs[USERNAME_KEY] ?: "Guest" }  
        .stateIn(viewModelScope, SharingStarted.Lazily, "Guest")  
  
    fun updateUsername(newName: String) {  
        viewModelScope.launch {  
            context.dataStore.edit { prefs ->  
                prefs[USERNAME_KEY] = newName  
            }  
        }  
    }  
}
```

UI Observes ViewModel State

```
@Composable
fun UserScreen(viewModel: UserViewModel) {
    val username by viewModel.username.collectAsState()

    Column {
        Text("Hello, $username")
        Button(onClick = { viewModel.updateUsername("Ahmed") }) {
            Text("Update Name")
        }
    }
}
```

- Clean separation of UI and data handling.

Validation and Error Handling

- Always validate data **before** writing to DataStore.

Example (Validation & Error Handling):

```
fun updateAge(newAge: Int) {  
    if (newAge > 0) {  
        viewModelScope.launch {  
            try {  
                context.dataStore.edit { prefs ->  
                    prefs[AGE_KEY] = newAge  
            }  
            } catch (e: IOException) {  
                Log.e("DataStore", "Error writing data: ${e.message}")  
            }  
        }  
    } else {  
        Log.e("Validation", "Invalid age")  
    }  
}
```

Using Proto DataStore (Advanced)

Step 1: Define Proto Schema (**user_prefs.proto**)

```
syntax = "proto3";  
  
option java_package = "com.example.app";  
option java_multiple_files = true;  
  
message UserPreferences {  
    string username = 1;  
    int32 age = 2;  
}
```

Set Up Proto DataStore

- Add dependency:

```
implementation "androidx.datastore:datastore:1.0.0"  
implementation "com.google.protobuf:protobuf-javalite:3.21.12"
```

- Initialize Proto DataStore:

```
val Context.userPreferencesStore by datastore(  
    fileName = "user_prefs.pb",  
    serializer = UserPreferencesSerializer  
)
```

Reading/Writing Proto DataStore

Writing data:

```
context.userPreferencesStore.updateData { prefs ->
    prefs.toBuilder()
        .setUsername("Alice")
        .setAge(28)
        .build()
}
```

Reading data:

```
val prefsFlow: Flow<UserPreferences> = context.userPreferencesStore.data
```


- **Avoid write conflicts:** serialize updates using `updateData()`.
- Use appropriate coroutine contexts (**IO** context recommended).
- Collect Flow efficiently (use `collectAsStateWithLifecycle()` if needed).

Example of efficient UI collection:

```
val userPrefs by context.userPreferencesStore.data
    .collectAsState(initial = UserPreferences.getDefaultInstance())
```

Common Mistakes to Avoid

- Frequent writes (batch updates instead).
- Ignoring potential IO exceptions (handle gracefully).
- Not providing default values or handling nulls.

Quiz Time

Key Takeaways

- Prefer DataStore for robust, async, and type-safe data storage.
- Use Preferences DataStore for simplicity; Proto DataStore for structure.
- Manage DataStore operations through ViewModel.
- Observe data reactively and handle errors proactively.

THANK YOU